COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithms	**SECTION: CPE21S3**	
DATE PERFORMED:	7/28/26	**DATE SUBMITTED:**	7/28/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Array-Based vs STL Stack Implementations

1. PROCEDURE OUTPUT

main.cpp

ILOA > main.cpp > main()

```cpp
#include <iostream>
#include <stack>

int main(){
    std::stack<int> newStack; //created an object

    std::cout<<"Is the stack empty?" << newStack.empty() << std::endl;

    //insert a 3 elemnt:
    newStack.push(3);
    std::cout<<"Top of stack: " << newStack.top()<< std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: " << newStack.top()<< std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: " << newStack.top()<< std::endl;

    std::cout<<"Is the stack empty? " << newStack.empty()<< std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);


    std::cout<<"Size of the stack is: " << newStack.size()<< std::endl;

    //swap?
    std::stack<int> stack1;
    std::stack<int> stack2;

    stack1.push(4);
    stack2.push(5);

    std::cout<<"Stack 1: "<<stack1.top()<< std::endl;
    std::cout<<"Stack 2: "<<stack2.top()<< std::endl;

    stack1.swap(stack2);
    std::cout<<"Stack 1: "<<stack1.top()<< std::endl;
    std::cout<<"Stack 2: "<<stack2.top()<< std::endl;

    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\CPE_Urbano\HOA4p1>  & 'c:\
nwnqxvz.k2l' '--stdout=Microsoft-MIEngine-Out-miu3c1mm
nterpreter=mi'
Is the stack empty?1
Top of stack: 3
Top of stack: 2
Top of stack: 1
Is the stack empty? 0
Size of the stack is: 6
Stack 1: 4
Stack 2: 5
Stack 1: 5
Stack 2: 4
PS C:\Users\TIPQC\Documents\CPE_Urbano\HOA4p1>
```

Code Analysis:
In this code, we have used the library named stack which has most of the built in functions for stack. Push() is used to add values to the stack, Pop() is used to remove the current top of the stack and replace it with the next one. We have used top() to print the top of the stack and we can see that it changes as we add another value to it. Size() is used to check the size of the stack and we have also used swap() to swap the values of the stacks.

ILO B:

main.cpp ILOA | main.cpp ILO B

ILO B > main.cpp > displayAll()

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;


//prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
void displayAll();

int main(){
    int choice;
    std::cout<<"Enter the number of maximum elments for the new stack (max:100): ";
    std::cin>> elements;

    while (true){
        std::cout<<"Stack Operations: "<< std::endl;
        std::cout << "1. PUSH \n2. POP \n3. Top \n4. isEmpty \n5. isFull \n6. displayAll"<< std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << isEmpty()<<std::endl;
            break;
            case 5: std::cout<< isFull()<< std::endl;
            case 6: displayAll();
            break;
            default: std::cout << "Invalid Choice" << std::endl;
            break;
        }
    }

    return 0;
}

```

Urbano_Seatwork4_1_ILOA.cpp | Urbano_Seatwork4_1_ILOB-1.cpp

ILO B > Urbano_Seatwork4_1_ILOB-1.cpp > push()

```
16  int main(){
45      return 0;
46  }
47
48  bool isEmpty(){
49      //top is -1
50      if (top == -1) return true;
51      return false;
52  }
53
54  bool isFull(){
55      if (top >= elements -1){
56          std::cout << "Stack Overflow" << std::endl;
57          return true;
58      }
59      return false;
60  }
61  void push(){
62
63      // check if stack full, if yes return an error
64      if (isFull()) return;
65
66      // ask the user for a value to be pushed into the stack
67      std::cout << "What number you want to add to the stack?: ";
68      std::cin >> newData;
69
70      //store inside the stack
71      stack[++top] = newData;
72  }
73  void pop(){
74
75      //if stack is empty
76      if(isEmpty()) return;
77
78      //display the element to be popped:
79      std::cout<<"Popping: "<< stack[top] << std::endl;
80
81      //decrement top
82      top--;
83  }
84  void Top(){
85      //error checking
86      if(isEmpty()) return;
87
88      //display top element
89      std::cout<<"The top element is: "<< stack[top] << std::endl;
90  }
91  void displayAll(){
```

```
92
93      //display all elements
94      std::cout <<"Printing Elements: " << std::endl;
95
96      for(int i = top; i >= 0; i--){
97          std::cout<< stack[i] << std::endl;
98      }
99
100 }
```

Output:

```
PS C:\Users\TIPQC\Documents\CPE_Urbano\HOA4p1>  & 'c:\Users\TIPQC\.vscode\extensions\
v3k2gwu.zdc' '--stdout=Microsoft-MIEngine-Out-bp5iuh14.0ub' '--stderr=Microsoft-MIEng
nterpreter=mi'
Enter the number of maximum elments for the new stack (max:100): 3
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
1
What number you want to add to the stack?: 3
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
1
What number you want to add to the stack?: 2
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
1
What number you want to add to the stack?: 1
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
2
Popping: 1
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
```

```
PS C:\Users\TIPQC\Documents\CPE_Urbano\HOA4p1>  & 'c:\Users\TIP
v3k2gwu.zdc' '--stdout=Microsoft-MIEngine-Out-bp5iuh14.0ub' '--
nterpreter=mi'
4. isEmpty
5. isFull
6. displayAll
7. Exit
2
Popping: 1
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
6
Printing Elements:
2
3
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
4
0
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
5
0
Printing Elements:
2
3
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
```

```
PS C:\Users\TIPQC\Documents\CPE_Urbano\HOA4p1>  & 'c:\Users\TIPQC\.v
v3k2gwu.zdc' '--stdout=Microsoft-MIEngine-Out-bp5iuh14.0ub' '--stder
nterpreter=mi'
0
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
5
0
Printing Elements:
2
3
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
6
Printing Elements:
2
3
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
```

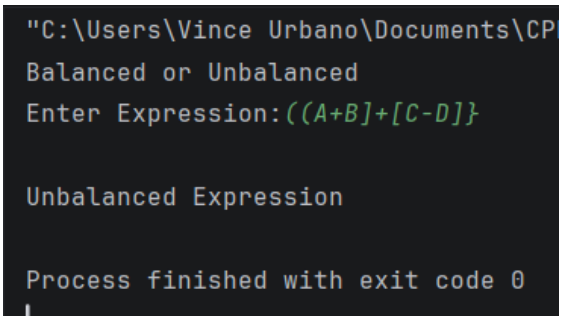
```
PS C:\Users\TIPQC\Documents\CPE_Urbano\HOA4p1>  & 'c:\Users\TIPQC\.vscode\extensic
bcbynar.mmu' '--stdout=Microsoft-MIEngine-Out-0nfivvrk.tja' '--stderr=Microsoft-MI
nterpreter=mi'
Enter the number of maximum elments for the new stack (max:100): 2
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
1
What number you want to add to the stack?: 1
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
1
What number you want to add to the stack?: 2
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. displayAll
7. Exit
1
Stack Overflow
```

Code Analysis:
In this file, instead of importing the stack library of stack, we have created our own functions which have the same functions as the library. isEmpty is still used to check if the stack has values or it does not have, if it has a value it will return true otherwise false. isFull will be used to check if our current stack has reached its maximum capacity and Stack Overflow will print if it does. Push function is still used to add values to the stack and each value added becomes the new top. Pop is used to remove the current top and replaces it with the next value. Top is used to print the value of the top. The displayAll function is responsible for printing all the values starting from the head.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL
Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Expression	Valid?	Output	Analysis
(A+B)+(C-D)	Yes	"C:\Users\Vince Urbano\Documents\CPE Balanced or Unbalanced Enter Expression:(A+B)+(C-D) Balanced Expression	This expression is considered a balanced expression because the program has checked that the opening delimiters match the amount of closing delimiters.(A+B) has both opening and closing the same as (C-D)
((A+B)+(C-D)	No	"C:\Users\Vince Urbano\Docume Balanced or Unbalanced Enter Expression:((A+B)+(C-D) Unbalanced Expression	The expression was considered as unbalanced because the program has checked that there was an excess of opening delimiter on the code making the equation unbalanced.
((A+B)+[C-D])	Yes	Balanced or Unbalanced Enter Expression:((A+B)+[C-D]) Balanced Expression	It is considered a valid expression because the opening delimiters have matching closing delimiters.

((A+B]+[C-D]}	No		This code was considered as an unbalanced expression because the opening and closing delimiters do not match the closing delimiters.

Code:

main (2).cpp × SuppList.h

```cpp
#include <iostream>
#include "SuppList.h"
#include <string>
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;


//prototyping
bool isEmptyArr();
bool isFullArr();
void pushArr();
void popArr();
void TopArr();
void displayAllArr();
void checkDelimiterArr();

int main(){

    std::string choice;
    std::cout << "Balanced or Unbalanced" << std::endl;
    std::cout << "Enter Expression: ";
    std::cin >> choice;

    bool isBalanced = true;

    for (size_t i = 0; i < choice.length(); i++) {
        char ch = choice[i];
```

```
        if (ch == '(' || ch == '{' || ch == '[') {
            push<char>(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty<char>()) {
                isBalanced = false;
                break;
            }

            char head = peek<char>();

            if ((head == '(' && ch == ')') ||
                (head == '{' && ch == '}') ||
                (head == '[' && ch == ']')) {
                pop<char>();
                } else {
                    isBalanced = false;
                    break;
                }
        }
    }

    if (isBalanced && isEmpty<char>()) {
        std::cout << "Balanced Expression" << std::endl;
```

```
        std::cout << "Balanced Expression" << std::endl;
    } else {
        std::cout << "Unbalanced Expression" << std::endl;
    }
    return 0;
}

bool isEmptyArr(){
    //top is -1
    if (top == -1) return true;
    return false;
}

bool isFullArr(){
    if (top >= elements -1){
        std::cout << "Stack Overflow" << std::endl;
        return true;
    }
    return false;
}
void pushArr(){

    if (isFullArr()) return;

    std::cout << "What number you want to add to the stack?: ";
    std::cin >> newData;
```

```
80      stack[++top] = newData;
81  }
82  void popArr(){
83
84      //if stack is empty
85      if(isEmptyArr()) return;
86
87      //display the element to be popped:
88      std::cout<<"Popping: "<< stack[top] << std::endl;
89
90      //decrement top
91      top--;
92  }
93  void TopArr(){
94      //error checking
95      if(isEmptyArr()) return;
96
97      //display top element
98      std::cout<<"The top element is: "<< stack[top] << std::endl;
99  }
100 void displayAllArr(){
101     //error checking
102     if(isEmptyArr()) return;
103
104     //display all elements
105     std::cout <<"Printing Elements: " << std::endl;
```

```
107     for(int i = top; i >= 0; i--){
108         std::cout<< stack[i] << std::endl;
109     }
110
111 }
112
```

SuppList.h:

main (2).cpp | SuppList.h

```cpp
#ifndef listStack_h
#define listStack_h

template <typename T>
class Node{
    public:
    T data;
    Node<T>* next = nullptr;
};

template<typename T>
Node<T>* head = nullptr;

template <typename T>
void push(T newData){
    //Create a new Node
    Node<T>* newNode = new Node<T>;
    //assign the date to the newNode
    newNode->data = newData;
    //point the new Node to the current top
    newNode->next = head<T>;
    //assign the new Top to the newNode
    head<T> = newNode;

}

template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
```

```
30  }
31
32  template <typename T>
33  T pop(){
34      //if stack is empty error catch:
35      if(isEmpty<T>()){
36          std::cout<< "Stack Underflow"<<std::endl;
37      }
38      //saving a current top node to a temporary variable;
39      Node<T>* temp = head<T>;
40      //store the data to a temp variable
41      T tempVal = temp->data;
42      //move the head to next node
43      head<T> = head<T>->next;
44      //free up the memory
45      delete temp;
46      //return the temporary variable data
47      return tempVal;
48  }
49
50  template <typename T>
51  T peek() {
52      if (isEmpty<T>()) {
53          return T();
54      }
55      return head<T>->data;
56  }
```

```
57
58  template <typename T>
59  void Top(){
60      if(isEmpty<T>()) return;
61      std::cout << "The top is: "<< head<T>->data <<std::endl;
62  }
63
64  template <typename T>
65  void displayAll(){
66      //check if empty
67      if(head<T> == nullptr){
68          std::cout<< "The list is empty"<<std::endl;
69          return;
70      }
71      //print
72      while(head<T>!= nullptr){
73          std::cout<<head<T>->data<<std::endl;
74          head<T> = head<T>->next;
75      }
76  }
77
78  #endif #ifndef listStack_h
```

Code Analysis:

In this program, we have created several functions for the header and main file. Both of them have the same functions which are push, isEmpty, top, pop, peek, and displayAll. The function of push is to add new data and this data is the new head of the stack. isEmpty is used as a checker of the stack, it checks if the stack is empty. Top is responsible for displaying the top of the stack. Pop is used to remove the data from the stack. Lastly, display all is used to print all of the values inside the stack. Moving to the int main, we can notice that we have a function named peek(). Peek is responsible for inspecting the delimiters, if the closing delimiter was detected the pop function will be called and remove the data. The for loop is responsible for checking every single value that was inputted by the user. We can see that we have several conditional statements which are responsible for detecting and making the decision for popping and if the expression is balanced or not.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance and Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI to assist me with debugging my current codes**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.